Métodos de Análisis de Datos 1

Clase 3: Fuentes de Datos y Carga

Universidad Nacional del Sur

Contenidos de hoy

1. Fuentes de datos
2. Carga de datos en Python
3. Carga de datos en R
4. Exploración inicial
5. Resumen

¿De dónde vienen los datos?

Los datos pueden provenir de múltiples fuentes:

Archivos

- CSV, TSV
- Excel (XLSX, XLS)
- JSON
- XML

Bases de datos

- SQLite
- PostgreSQL
- MySQL/MariaDB
- SQL Server

APIs Web

- REST APIs
- GraphQL
- Autenticación (API keys)

Web Scraping

- HTML parsing
- Extracción de tablas
- BeautifulSoup, rvest

CSV: El formato universal

Comma-Separated Values

Valores separados por comas (o punto y coma, tabulaciones...)

Ventajas:

- Simple y ligero
- Compatible con todo
- Fácil de inspeccionar
- Portable

Desventajas:

- No preserva tipos
- Problemas con delimitadores
- Sin múltiples hojas
- Sin compresión

Ejemplo

```
nombre,edad,ciudad
Ana,25,Bahía Blanca
Luis,30,Buenos Aires
```

Bases de datos relacionales

¿Por qué usar bases de datos?

- Datos grandes (GB/TB)
- Consultas complejas
- Múltiples usuarios simultáneos
- Integridad y transacciones
- Indexación y optimización

Bases de datos relacionales

¿Por qué usar bases de datos?

- Datos grandes (GB/TB)
- Consultas complejas
- Múltiples usuarios simultáneos
- Integridad y transacciones
- Indexación y optimización

SQL: Structured Query Language

Lenguaje estándar para consultar bases de datos relacionales

```
SELECT * FROM ventas WHERE fecha >= '2024-01-01'
```

APIs: Acceso programático a datos

Application Programming Interface

Permite acceder a datos de servicios web de forma estructurada

Ejemplo: API del clima

`https://api.openweathermap.org/data/2.5/weather?q=BahiaBlanca&appid=TU_API_KEY`

Retorna JSON con temperatura, humedad, presión...

APIs: Acceso programático a datos

Application Programming Interface

Permite acceder a datos de servicios web de forma estructurada

Ejemplo: API del clima

`https://api.openweathermap.org/data/2.5/weather?q=BahiaBlanca&appid=TU_API_KEY`

Retorna JSON con temperatura, humedad, presión...

Ventajas

- Datos siempre actualizados
- Formato estructurado (JSON)
- Documentación oficial
- Rate limits (controlan acceso)

Web Scraping: Última opción

¿Cuándo usar web scraping?

Solo cuando NO hay API disponible

Proceso

1. Descargar HTML de la página
2. Inspeccionar estructura (DevTools)
3. Identificar selectores CSS/XPath
4. Extraer información
5. Limpiar y estructurar

Web Scraping: Última opción

¿Cuándo usar web scraping?

Solo cuando NO hay API disponible

Proceso

1. Descargar HTML de la página
2. Inspeccionar estructura (DevTools)
3. Identificar selectores CSS/XPath
4. Extraer información
5. Limpiar y estructurar

Consideraciones legales

- Revisar términos de servicio
- Respetar robots.txt
- Rate limiting (no saturar servidor)

Pandas: La herramienta principal

Pandas

Librería principal para manipulación de datos en Python

Cargar CSV:

```
import pandas as pd

# B sico
df = pd.read_csv('datos.csv')

# Con opciones
df = pd.read_csv(
    'datos.csv',
    sep=',',                # Delimitador
    encoding='utf-8',       # Codificaci n
    na_values=['NA', ''],   # Valores NA
    parse_dates=['fecha']   # Parsear fechas
)
```

Otros formatos en Pandas

Excel:

```python
# Una hoja
df = pd.read_excel(
    'datos.xlsx',
    sheet_name='Hoja1'
)

# Todas las hojas
todas = pd.read_excel(
    'datos.xlsx',
    sheet_name=None
)
```

JSON:

```python
# JSON simple
df = pd.read_json('datos.json')

# JSON anidado
import json
with open('datos.json') as f:
    data = json.load(f)
df = pd.json_normalize(data)
```

Conectar a bases de datos

SQLite (local):

```
import sqlite3
import pandas as pd

conn = sqlite3.connect('base.db')

query = "SELECT * FROM ventas WHERE fecha >= '2024-01-01'"
df = pd.read_sql_query(query, conn)

conn.close()
```

Conectar a bases de datos

SQLite (local):

```
import sqlite3
import pandas as pd

conn = sqlite3.connect('base.db')

query = "SELECT * FROM ventas WHERE fecha >= '2024-01-01'"
df = pd.read_sql_query(query, conn)

conn.close()
```

PostgreSQL/MySQL (con SQLAlchemy):

```
from sqlalchemy import create_engine

engine = create_engine('postgresql://user:pass@localhost/db')
df = pd.read_sql_query("SELECT * FROM tabla", engine)
```

Acceder a APIs en Python

```python
import requests
import pandas as pd

# Request a API
url = "https://api.ejemplo.com/datos"
headers = {'Authorization': 'Bearer TU_TOKEN'}
response = requests.get(url, headers=headers)

# Verificar que fue exitoso
if response.status_code == 200:
    data = response.json()
    df = pd.DataFrame(data)
else:
    print(f"Error: {response.status_code}")
```

Códigos HTTP importantes

200 — OK | 401 — No autorizado | 404 — No encontrado | 500 — Error del servidor

Tidyverse: Ecosistema de R

readr

Parte de tidyverse, carga rápida y eficiente

Cargar CSV:

```
library(readr)

# Básico
df <- read_csv('datos.csv')

# Con opciones
df <- read_csv(
  'datos.csv',
  col_types = cols(
    edad = col_integer(),
    fecha = col_date(format = "%Y-%m-%d")
  ),
  na = c("NA", "", "N/A")
```

Otros formatos en R

Excel:

```r
library(readxl)

# Una hoja
df <- read_excel(
  'datos.xlsx',
  sheet = 'Hoja1'
)

# Ver hojas
excel_sheets('datos.xlsx')
```

JSON:

```r
library(jsonlite)

# Cargar JSON
data <- fromJSON('datos.json')

# A data frame
df <- as.data.frame(data)
```

Conectar a bases de datos en R

SQLite:

```
library(DBI)
library(RSQLite)

conn <- dbConnect(RSQLite::SQLite(), "base.db")
df <- dbGetQuery(conn, "SELECT * FROM ventas")
dbDisconnect(conn)
```

Conectar a bases de datos en R

SQLite:

```
library(DBI)
library(RSQLite)

conn <- dbConnect(RSQLite::SQLite(), "base.db")
df <- dbGetQuery(conn, "SELECT * FROM ventas")
dbDisconnect(conn)
```

PostgreSQL:

```
library(RPostgres)

conn <- dbConnect(
  RPostgres::Postgres(),
  dbname = 'mi_base',
  host = 'localhost',
  user = 'usuario',
  password = 'clave'
```

Acceder a APIs en R

```r
library(httr)
library(jsonlite)

# Request
url <- "https://api.ejemplo.com/datos"
response <- GET(url,
                add_headers(Authorization = "Bearer TOKEN"))

# Verificar status
if (status_code(response) == 200) {
  data <- content(response, as = "text")
  df <- fromJSON(data)
} else {
  print(paste("Error:", status_code(response)))
}
```

Siempre explorar después de cargar

Python:

```python
# Dimensiones
df.shape  # (filas, cols)

# Primeras filas
df.head()

# Info general
df.info()

# Estadísticas
df.describe()

# Columnas
df.columns

# Tipos
```

R:

```r
# Dimensiones
dim(df)

# Primeras filas
head(df)

# Estructura
str(df)

# Estadísticas
summary(df)

# Columnas
names(df)

# Tipos
```

Checklist post-carga

Después de cargar datos, SIEMPRE verificar:

1. **Dimensiones correctas:** ¿Tiene sentido el número de filas y columnas?
2. **Tipos de datos:** ¿Las columnas tienen el tipo correcto?
3. **Valores faltantes:** ¿Hay NAs? ¿Cuántos?
4. **Nombres de columnas:** ¿Son claros y consistentes?
5. **Primeras y últimas filas:** ¿Los datos se ven bien?
6. **Rangos:** ¿Los valores están en rangos esperados?

Regla de oro

Nunca confíes en los datos crudos. Siempre verificar antes de analizar.

Resumen de la clase

- Los datos vienen de **múltiples fuentes**: archivos, DBs, APIs, web
- **CSV** es universal pero tiene limitaciones
- Las **bases de datos** son ideales para datos grandes y consultas complejas
- Las **APIs** proveen datos estructurados y actualizados
- El **web scraping** es último recurso (considerar legalidad)
- **Pandas (Python)** y **readr/tidyverse (R)** son las herramientas principales
- **Siempre explorar** después de cargar datos

Próxima clase

Calidad de datos: valores faltantes, duplicados, inconsistencias

Tarea

1. Descargar un dataset en formato CSV (ej: de Kaggle)
2. Cargarlo en Python y R
3. Hacer exploración inicial en ambos lenguajes
4. Documentar en un notebook:
   - Dimensiones
   - Tipos de datos
   - Primeras 10 filas
   - Estadísticas descriptivas
   - ¿Hay valores faltantes?
5. Subir notebook a GitHub